

Exercise 4 - Comparing parallel for with parallel for collapse(2)

Source files : exercise04.c (original version)

exercise04_collapse.c (version using collapse(2))

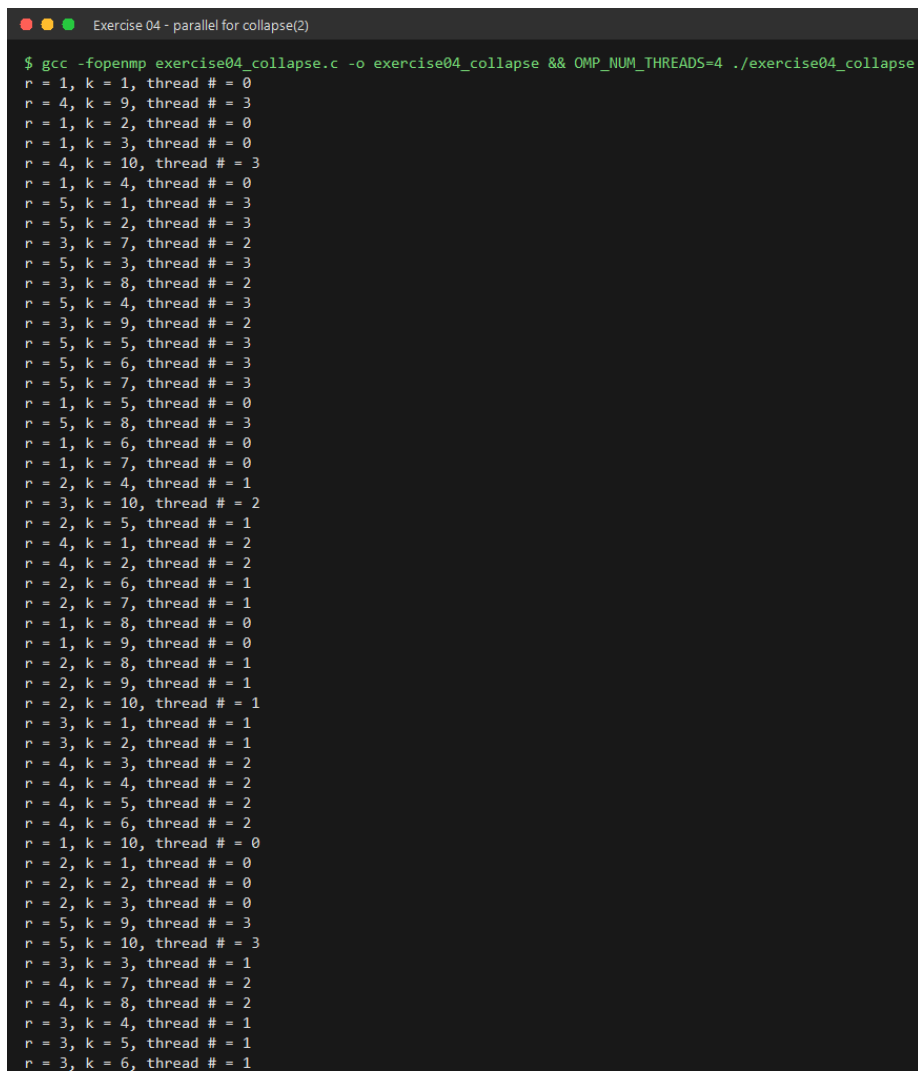
Build and execute

```
gcc -fopenmp exercise04.c -o exercise04
```

```
OMP_NUM_THREADS=4 ./exercise04
```

```
gcc -fopenmp exercise04_collapse.c -o exercise04_collapse
```

```
OMP_NUM_THREADS=4 ./exercise04_collapse
```



```
Exercise 04 - parallel for collapse(2)

$ gcc -fopenmp exercise04_collapse.c -o exercise04_collapse && OMP_NUM_THREADS=4 ./exercise04_collapse
r = 1, k = 1, thread # = 0
r = 4, k = 9, thread # = 3
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 4, k = 10, thread # = 3
r = 1, k = 4, thread # = 0
r = 5, k = 1, thread # = 3
r = 5, k = 2, thread # = 3
r = 3, k = 7, thread # = 2
r = 5, k = 3, thread # = 3
r = 3, k = 8, thread # = 2
r = 5, k = 4, thread # = 3
r = 3, k = 9, thread # = 2
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 5, k = 7, thread # = 3
r = 1, k = 5, thread # = 0
r = 5, k = 8, thread # = 3
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 2, k = 4, thread # = 1
r = 3, k = 10, thread # = 2
r = 2, k = 5, thread # = 1
r = 4, k = 1, thread # = 2
r = 4, k = 2, thread # = 2
r = 2, k = 6, thread # = 1
r = 2, k = 7, thread # = 1
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 2, k = 8, thread # = 1
r = 2, k = 9, thread # = 1
r = 2, k = 10, thread # = 1
r = 3, k = 1, thread # = 1
r = 3, k = 2, thread # = 1
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 4, k = 6, thread # = 2
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 5, k = 9, thread # = 3
r = 5, k = 10, thread # = 3
r = 3, k = 3, thread # = 1
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 3, k = 4, thread # = 1
r = 3, k = 5, thread # = 1
r = 3, k = 6, thread # = 1
```

1. #pragma omp parallel for

Here only the outer loop over r (1..5) is shared out. Its five iterations are handed to the four threads, and whichever thread owns a given r executes the complete inner loop over k (1..10) for it.

thread	r values	iterations
--------	------------	------------

-----	-----	-----
-------	-------	-------

0	$r = 1, r = 2$	20
---	----------------	----

1	$r = 3$	10
---	---------	----

2	$r = 4$	10
---	---------	----

3	$r = 5$	10
---	---------	----

Since 5 is not a multiple of 4 the distribution is uneven: thread 0 ends up with double the work of every other thread.

2. #pragma omp parallel for collapse(2)

With collapse(2) the two nested loops are flattened into a single iteration space containing $5 \times 10 = 50$ (r,k) pairs, and it is these pairs that get shared between the threads.

thread	(r,k) pairs	iterations
--------	---------------	------------

-----	-----	-----
-------	-------	-------

0	$(1,1) \dots (2,3)$	13
---	---------------------	----

1	$(2,4) \dots (3,6)$	13
---	---------------------	----

2	(3,7) ... (4,8)	12
3	(4,9) ... (5,10)	12

The boundaries between threads no longer line up with the r values
(threads 0 and 1 both receive part of $r = 2$), but in return the
workload is now even at 13/13/12/12 iterations per thread.

Exercise 04 - parallel for

```
$ gcc -fopenmp exercise04.c -o exercise04 && OMP_NUM_THREADS=4 ./exercise04
```

```
r = 3, k = 1, thread # = 1
r = 5, k = 1, thread # = 3
r = 3, k = 2, thread # = 1
r = 1, k = 1, thread # = 0
r = 3, k = 3, thread # = 1
r = 3, k = 4, thread # = 1
r = 5, k = 2, thread # = 3
r = 5, k = 3, thread # = 3
r = 4, k = 1, thread # = 2
r = 5, k = 4, thread # = 3
r = 4, k = 2, thread # = 2
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 1, k = 2, thread # = 0
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 1, k = 3, thread # = 0
r = 3, k = 5, thread # = 1
r = 5, k = 7, thread # = 3
r = 3, k = 6, thread # = 1
r = 5, k = 8, thread # = 3
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 4, k = 6, thread # = 2
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 4, k = 9, thread # = 2
r = 5, k = 9, thread # = 3
r = 5, k = 10, thread # = 3
r = 4, k = 10, thread # = 2
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
```